HACKATHON CHALLENGE BRIEF

AI-Powered Interactive Learning Assistant

with AI Mock Interview Extension

Build a hosted, context-aware learning system that understands video content, PDFs, PPTs and learner progress - then extend it into an intelligent mock interview experience.

GENERATIVE AI | RAG | AGENTIC AI | EDTECH | AWS

THE GOAL

Build a production-oriented working system, not just a UI prototype. Teams may use open-source components as a starting point and are encouraged to optimize for accuracy, latency, cost and real-world integration.

Challenge Snapshot

Core Experience Watch -> Pause -> Ask -> Understand -> Continue	**Context Sources** Video transcript, PDF, PPT, notes, code and course resources
Key Intelligence Timestamp-aware retrieval with grounded answers and citations	**Final Output** Hosted app, source code, API, architecture, evaluation and deployment docs

1. Problem Overview

Most online learning platforms remain one-directional. When learners do not understand a concept, they often leave the platform to search externally, inspect documents manually, or wait for a mentor.

Build an AI Learning Assistant integrated with a video-learning experience. A learner should be able to pause a lecture at any point and ask a question that is answered from the correct learning context.

> **Example**
>
> A 60-minute lecture is paused at 18:42. The learner asks, "Why did the instructor use this method?" The AI should use the current video, transcript up to 18:42, permitted PDFs/PPTs/resources and conversation history to answer accurately.

2. Core Requirement: Timestamp-Aware AI

The system must understand how much of the video has already been watched. If the learner is at 18:42, the AI should primarily reason from content between 00:00 and 18:42 plus explicitly permitted supporting resources.

```
VIDEO: 60 MINUTES
      |
CURRENT POSITION: 18:42
      |
00:00 - 18:42 + PERMITTED RESOURCES
      |
RETRIEVE -> REASON -> CITE -> ANSWER
```

The system should avoid unnecessarily revealing material taught later in the video. Timestamp awareness will be a major evaluation criterion.

3. Multi-Resource Understanding

The solution should understand more than the transcript. Teams may combine video and supporting learning resources into a unified retrieval layer.

Video Transcript segments with timestamps and metadata	**Documents** PDFs, PPTs, notes and text resources
Code / Assets Code files or structured course resources where relevant	**Knowledge Layer** Vector search, hybrid search, knowledge graph, GraphRAG or agentic retrieval

Technology choices are open, but teams should explain why their architecture improves retrieval quality, grounding, latency, scalability or cost.

4. Open-Source Starting Point

> **OPEN-SOURCE IS ALLOWED**
>
> Teams do not need to build every component from scratch. They may use suitable open-source projects, libraries, models, video players, LMS components, RAG frameworks, vector databases, transcription tools, PDF/PPT parsers or publicly available sample learning content.

A practical starting dataset can be as small as:

```
1 SAMPLE EDUCATIONAL VIDEO
+ 1 PDF
+ 1 PPT
+ SUPPORTING NOTES
      |
BUILD THE COMPLETE END-TO-END EXPERIENCE
```

The focus is on system design, AI intelligence, integration, optimization and deployment - not reinventing basic infrastructure.

5. Expected Learner Experience

The assistant should ideally support questions and actions such as:

- Explain this concept in simpler words.
- Give me an example or analogy.
- What did the instructor mean here?
- Where was this discussed in the video?
- Is this explained in the PDF or PPT?
- Summarize what I have learned so far.
- Generate questions or a short quiz from content covered so far.
- Compare two concepts.

6. Grounding and Hallucination Control

Answers should show where information came from. Citations may reference video timestamps, PDF pages or PPT slides.

> **GROUNDING EXAMPLE**
>
> Answer: Attention helps a model identify which tokens are more relevant.
> Sources: Video 14:21-16:08 | PDF Page 7 | PPT Slide 11

If the answer is not available in permitted resources, the system should say so instead of inventing an answer. Correct refusal is better than confident hallucination.

7. Production Expectations

This challenge is intended to produce a working, production-oriented system. A simple chatbot connected directly to an LLM API is not sufficient.

- Hosted and publicly accessible
- API-driven and modular
- Secure and documented
- Scalable for concurrent learners
- Designed for integration with an existing LMS
- Uses clean configuration, logging and error handling

```
VIDEO PLAYER
   |
FRONTEND -> BACKEND API
            |
SESSION / CONVERSATION LAYER
            |
RETRIEVAL -> VECTOR / GRAPH KNOWLEDGE
            |
LLM -> GROUNDED ANSWER + SOURCES
```

8. Cloud and AWS Preference

An AWS-first architecture is encouraged because the final system may need to integrate with AWS-based infrastructure. Possible services include S3, Bedrock, Transcribe, OpenSearch, DynamoDB, ECS/Lambda, API Gateway, CloudWatch and Neptune.

There is no mandatory cloud restriction. Teams may deploy anywhere, provided the final system is accessible and the architecture is production-oriented.

9. Optimization Requirements

Teams should optimize for:

Accuracy Relevant retrieval, correct citations and low hallucination	**Latency** Fast responses without sending excessive context
Cost Efficient embeddings, model calls and cloud resources	**Scale** Concurrent users and growth from pilot to large learner volumes

For example, the system should not send an entire one-hour transcript to the LLM for every question. Efficient retrieval is expected.

10. Evaluation and Testing

Teams should test at least the following scenarios:

- Question about content discussed before the current timestamp.
- Question about content covered later in the lecture.
- Question answered by a PDF or PPT.
- Question not present in any resource.
- Multiple connected follow-up questions.
- Verification that cited timestamps/pages/slides actually support the answer.

Recommended metrics include retrieval accuracy, citation accuracy, hallucination rate, response latency and cost per query.

11. Mandatory Deliverables

01	Hosted working application
02	Source code repository
03	Architecture diagram
04	README / setup documentation
05	API documentation
06	Sample dataset
07	Evaluation results
08	Short demo video
09	Deployment instructions

SUBMISSION RULE

Localhost-only submissions will not be considered complete.

12. Judging Criteria - 100 Marks

Category	Marks
Core Functionality & Learning Experience	20
RAG Accuracy & Grounding	20
Timestamp Awareness	15
Technical Architecture	15
Production Readiness & Deployment	10
Performance & Cost Optimization	10
Code Quality, Security & Documentation	10

13. Advanced Extension: AI Mock Interview System

Teams that complete the main challenge may extend the platform with an AI-based mock interview system. Interviews may be personalized using the candidate resume, target role, job description, course completed, skill level and interview type.

> **DYNAMIC, NOT STATIC**
>
> The interviewer should ask follow-up questions based on the candidate's previous answer rather than simply playing a fixed list of questions.

Example flow:

AI: Tell me about your machine-learning project.
CANDIDATE: I built a churn model using XGBoost.
AI: Why XGBoost instead of logistic regression?
CANDIDATE: ...
AI: How would you detect overfitting in that model?

14. Interview Output

After the interview, the system should generate explainable feedback rather than an arbitrary score.

Scorecard Overall score plus technical knowledge, communication, problem solving and answer structure	**Feedback** Strengths, improvement areas and specific preparation recommendations
Optional Voice Mode Speech-to-text and text-to-speech for a realistic interview experience	**Cost Awareness** Estimate the operating cost of a 10, 20 or 30-minute interview

15. 2X Reward Opportunity

EXTENDED REWARD

Teams delivering both a strong AI Learning Assistant and a genuinely functional AI Mock Interview System may be considered for a 2X reward, subject to technical quality and functional completeness. The extension must be a working system, not merely an extra page or static feature.

Final Goal

The objective is not to build another generic chatbot. The goal is an AI-native learning experience that keeps the learner inside one intelligent flow:

```
WATCH -> PAUSE -> ASK -> UNDERSTAND -> PRACTICE
                -> REVISE -> PREPARE -> INTERVIEW
```

ONE-LINE CHALLENGE

Build and deploy a timestamp-aware AI Learning Assistant that understands videos and supporting resources such as PDFs and PPTs, provides grounded contextual answers, and can be extended into an AI-powered mock interview platform.

